

Phase 2의 목표는 Codex가 `kdqe` Skill을 통해 자연어 질문을 SQL로 변환하고, SQLite를 조회한 후 결과를 자연어로 응답하는 것입니다. 즉, Codex가 KDQE의 "두뇌" 역할을 하도록 만드는 단계입니다.

🎯 Phase 2 목표

- 자연어 질문 → Codex가 SQL 생성 → SQLite 실행 → 자연어 응답 생성
- Codex가 `kdqe` Skill을 읽고 데이터베이스 스키마, VIP 정의, 질의 패턴을 이해하도록 함
- 모든 과정이 Codex 내부에서 자연스럽게 이루어지도록 Skill 설계

🚀 Phase 2 실행 계획 (상세 단계)

Step 0: 사전 준비 확인

- ☒ SQLite DB (`data/techshop.db`)에 4개 테이블(`customers`, `products`, `orders`, `order_items`)이 정상 로드 됨
- ☒ `.agent/skills/kdqe/SKILL.md` 파일이 존재함
- ☒ Codex가 `kdqe` Skill을 인식함 (`/skills` 목록에 나타남)

Step 1: `kdqe` Skill 내용 고도화 (가장 중요)

Codex가 데이터를 이해하려면 Skill에 다음 정보가 상세히 포함되어야 합니다:

`4-KDQEW.agentWskillsWkdqeWSKILL.md` 파일을 편집합니다.

```
---
name: kdqe
description: Unified schema and instructions for querying the TechShop e-commerce database.
triggers:
  - "VIP"
  - "고객"
  - "매출"
  - "주문"
  - "제품"
---

# KDQE: TechShop E-Commerce Database Query Skill

## 🇹🇷 데이터베이스 스키마

TechShop 데이터베이스는 다음 4개 테이블로 구성됩니다:

### 1. customers (고객)
| 컬럼명 | 타입 | 설명 |
|-----|-----|-----|
| id | INTEGER | PK |
| email | TEXT | 이메일 |
| name | TEXT | 고객명 |
| phone | TEXT | 전화번호 |
| grade | TEXT | 등급 (VIP, GOLD, SILVER, BRONZE) |
| point_balance | INTEGER | 포인트 잔액 |
| is_active | INTEGER | 활성 여부 (1=활성, 0=비활성) |
| created_at | TEXT | 가입일 (ISO 8601) |

### 2. products (제품)
```

```
| id | INTEGER | PK |
| name | TEXT | 제품명 |
| brand | TEXT | 브랜드 |
| sku | TEXT | 재고 관리 코드 |
| price | REAL | 판매가 |
| cost_price | REAL | 원가 |
| stock_qty | INTEGER | 재고 수량 |
| is_active | INTEGER | 판매 활성 여부 |
| created_at | TEXT | 등록일 |
```

3. orders (주문)

```
| 컬럼명 | 타입 | 설명 |
|-----|-----|-----|
| id | INTEGER | PK |
| order_number | TEXT | 주문 번호 |
| customer_id | INTEGER | 고객 ID (customers.id) |
| status | TEXT | 주문 상태 (pending, paid, shipped, delivered, cancelled) |
| total_amount | REAL | 총 주문 금액 |
| ordered_at | TEXT | 주문 일시 |
```

4. order_items (주문 상세)

```
| 컬럼명 | 타입 | 설명 |
|-----|-----|-----|
| id | INTEGER | PK |
| order_id | INTEGER | 주문 ID (orders.id) |
| product_id | INTEGER | 제품 ID (products.id) |
| quantity | INTEGER | 수량 |
| unit_price | REAL | 단가 |
| subtotal | REAL | 소계 (quantity * unit_price) |
```

🔍 비즈니스 규칙 (OKF 개념)

VIP 고객 정의

- `grade = 'VIP'` 인 고객
- 관련 질문: "VIP 고객 수", "VIP 고객 목록", "VIP 고객 평균 구매액"

활성 고객

- `is\_active = 1` 인 고객

최근 주문

- `ordered\_at >= date('now', '-7 days')` (최근 7일)

🗣️ 자연어 → SQL 변환 패턴

```
| 자연어 질문 예시 | 변환될 SQL |
|-----|-----|
```

```
| "VIP 고객은 몇 명이고, 누구야?" | `SELECT id, name, email FROM customers WHERE grade = 'VIP'` |
| "전체 고객 수는?" | `SELECT COUNT(*) FROM customers` |
| "가장 비싼 상품 3개" | `SELECT name, price FROM products ORDER BY price DESC LIMIT 3` |
| "카테고리별 평균 가격" | `SELECT category, AVG(price) FROM products GROUP BY category` |
| "최근 7일간 주문 목록" | `SELECT order_number, total_amount FROM orders WHERE ordered_at >= date('now', '-7 days')` |
| "VIP 고객의 평균 구매액" | `SELECT AVG(total_amount) FROM orders o JOIN customers c ON o.customer_id = c.id` |
```

📌 응답 형식

SQL 실행 결과는 다음과 같은 자연어 형식으로 응답해야 합니다:

- **집계 질문**: "VIP 고객은 총 5명입니다."
- **목록 질문**: "VIP 고객 목록: 1. 홍길동 (hong@test.kr), 2. 김철수 (kim@test.kr), ..."
- **비교/추세**: "가장 비싼 상품은 '삼성 비스포크 냉장고'로 2,150,000원입니다."

🛠️ SQL 실행 방법

1. 위 패턴을 참고하여 사용자 질문에 맞는 SQL을 생성합니다.
2. SQLite DB(`data/techshop.db`)에 대해 SQL을 실행합니다.
3. 결과를 위 응답 형식에 맞게 가공하여 출력합니다.
4. 만약 SQL 실행 오류가 발생하면, 오류 메시지를 해석하여 수정된 SQL을 재시도합니다.

⚠️ 주의사항

- `datetime` 함수는 SQLite 내장 함수 사용 (`date('now')`, `datetime('now')`)
- 테이블명과 컬럼명은 대소문자 구분 없음
- 문자열 비교는 작은따옴표(') 사용
- 집계 함수 사용 시 적절한 `GROUP BY` 필요

Step 2: `data_layer_query.py` 스크립트 구현 (선택적)

Codex가 직접 SQL을 생성하고 실행할 수도 있지만, **Python 스크립트로 실행을 위임**하면 더 안정적이고 로깅/디버깅이 용이합니다.

`scripts/data_layer_query.py` 파일을 생성합니다:

```
#!/usr/bin/env python3
"""
Data Layer Query Script - SQLite 질의 실행 및 결과 반환
Codex의 kdqe Skill에 의해 호출됨
"""

import sqlite3
import json
import sys
from pathlib import Path

DB_PATH = Path(__file__).parent.parent / "data" / "techshop.db"

def run_query(sql: str) -> list:
    """SQLite 쿼리 실행 및 결과 반환"""
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    try:
        cursor.execute(sql)
        results = cursor.fetchall()
```

```

        conn.close()
        return results
    except sqlite3.Error as e:
        conn.close()
        raise e

def format_results(results: list, headers: list = None) -> str:
    """결과를 자연어/표 형식으로 포맷팅"""
    if not results:
        return "⚠️ 결과가 없습니다."

    if not headers:
        # 첫 행의 길이로 헤더 자동 생성
        headers = [f"col_{i+1}" for i in range(len(results[0]))]

    # 표 형태로 출력 (Markdown 테이블)
    lines = ["| " + " | ".join(headers) + " |"]
    lines.append("| " + " | ".join(["---"] * len(headers)) + " |")
    for row in results:
        row_str = "| " + " | ".join(str(v) if v is not None else "" for v in row) + " |"
        lines.append(row_str)

    return "\n".join(lines)

def main():
    if len(sys.argv) < 2:
        print("Usage: python data_layer_query.py '<SQL query>'")
        sys.exit(1)

    sql = sys.argv[1]
    try:
        results = run_query(sql)
        print(format_results(results))
        print(f"\n📊 총 {len(results)}개 행")
    except sqlite3.Error as e:
        print(f"❌ SQL 오류: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Step 3: Codex에서 kdqe Skill 테스트

1.

Codex를 실행합니다:

```
cd D:\code\metaprogramming\KernelAgent\W4-KDQE
codex
```

2.

다음 자연어 질문을 입력합니다:

VIP 고객은 몇 명이고, 누구야?

3.

Codex가 `kdqe` Skill을 읽고 다음을 수행할 것으로 기대합니다:

- Skill에서 VIP 정의(`grade='VIP'`)를 찾음

- SQL 생성: `SELECT id, name, email FROM customers WHERE grade='VIP'`
- SQL 실행 (또는 `data_layer_query.py` 호출)
- 결과를 자연어로 응답

Step 4: 응답 예시

Codex가 생성할 응답 예시:

🔍 VIP 고객 조회 결과:
총 5명의 VIP 고객이 있습니다.

ID	이름	이메일
4	정민호	dave@test.kr
8	오세훈	hank@test.kr
12	강태영	leo@test.kr
16	문창호	peter@test.kr
20	황미소	tina@test.kr

📋 VIP 고객 목록: 정민호, 오세훈, 강태영, 문창호, 황미소

Step 5: 추가 질문 테스트

다양한 질문을 테스트하여 Skill의 완성도를 높입니다:

테스트 질문	기대 동작
"가장 비싼 상품 3개 알려줘"	<code>SELECT name, price FROM products ORDER BY price DESC LIMIT 3</code>
"전체 주문 수는?"	<code>SELECT COUNT(*) FROM orders</code>
"최근 7일간 주문 내역"	<code>SELECT * FROM orders WHERE ordered_at >= date('now', '-7 days')</code>
"VIP 고객의 평균 주문 금액은?"	<code>SELECT AVG(o.total_amount) FROM orders o JOIN customers c ON o.customer_id = c.id WHERE c.grade='VIP'</code>

Step 6: 오류 처리 및 디버깅

Codex가 잘못된 SQL을 생성하면:

1. SQLite 오류 메시지를 확인
2. Skill에 해당 패턴을 추가하거나 수정
3. Codex에게 "이 오류를 해결해봐"라고 지시

📁 Phase 2 완료 시 파일 구조

```
4-KDQE/
├── .agent/
│   ├── skills/
│   │   └── kdqe/
│   │       └── SKILL.md          # ✅ 고도화 완료
├── data/
│   └── techshop.db             # ✅ 4개 테이블 데이터
├── scripts/
│   └── data_layer_query.py      # ✅ SQL 실행 스크립트
└── AGENTS.md                   # ✅ Codex 진입점
```

✅ Phase 2 체크리스트

- ☐ kdqe/SKILL.md 에 스키마, VIP 정의, SQL 패턴 추가
- ☐ scripts/data_layer_query.py 생성 및 테스트
- ☐ Codex에서 kdqe Skill 인식 확인 (/skills 목록)
- ☐ "VIP 고객은 몇 명이고, 누구야?" 질의 테스트
- ☐ 추가 질문 5개 이상 테스트 및 응답 확인
- ☐ 오류 케이스(예: 존재하지 않는 컬럼) 처리 확인

💡 팁: Codex의 자동 SQL 생성 능력 활용

Codex는 이미 자연어를 SQL로 변환하는 능력이 뛰어납니다. 따라서 Skill에 모든 SQL 패턴을 미리 적을 필요는 없고, 스키마와 비즈니스 규칙(VIP 정의)만 명확히 알려주면 Codex가 스스로 적절한 SQL을 생성합니다. Skill은 "가이드북" 역할에 집중하세요.